

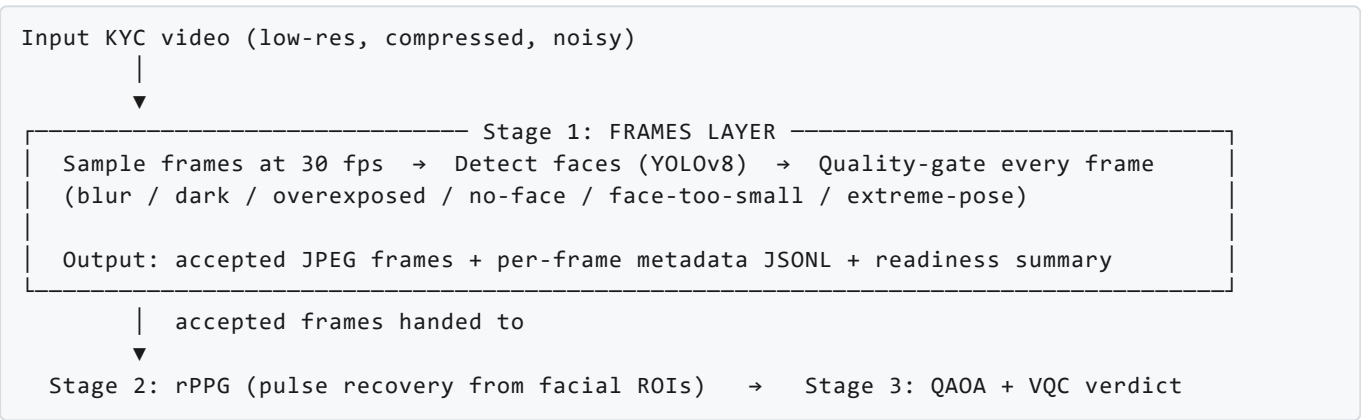
Frame Sampling & Quality Layer — Stage 1

Presentation Document — Deepfake Video Detection for KYC (rPPG + Hybrid Quantum ML)

Scope of this document: everything an examiner needs to understand, present, and defend the *frames layer* (Stage 1) of the project: its role, design, algorithms, thresholds, outputs, integration, and evidence. This is a document only — no code is changed.

Component: `WORKING/frame/` — Stage 1 of the three-stage pipeline.

1. One-Slide Summary



Why it exists: rPPG (remote photoplethysmography) recovers a person's pulse from subtle skin-color changes across frames. One blurry, dark, or face-less frame corrupts the pulse trace; garbage frames destroy the physiological evidence. Stage 1 is therefore the **signal-quality gate** that decides *which frames* are allowed to feed the physiology stage — it converts a raw video into a clean, ordered, well-timestamped frame sequence.

2. Position in the Overall System

Stage	Directory	Responsibility
1 — Frames (this layer)	<code>WORKING/frame/</code>	Frame sampling, YOLO face detection, quality filtering
2 — rPPG	<code>WORKING/RPPG/</code>	MediaPipe face ROIs → POS/CHROM pulse → 20 physiological features
3 — Quantum	<code>WORKING/quantum/</code>	QAOA feature selection (20→3) → hybrid VQC → P(real) → REAL / FAKE / UNCERTAIN

The end-to-end orchestrator `WORKING/run_pipeline.py` calls this layer as its first stage:

```
frames → rPPG → quantum → verdict (REAL / FAKE / UNCERTAIN)
```

- Stage 1 runs **automatically** inside `run_pipeline.py` (sample rate **30 fps**).
- Its accepted JPEGs and `frame_metadata.jsonl` are handed directly to `RPPGPipeline.process_frames()` (Stage 2).

- If Stage 1 has nothing to hand over (e.g., source is not a local file), the pipeline falls back to direct video read (`RPPGPipeline.process_video()`) and records `input_mode` in the result JSON.

3. Module Map (`WORKING/frame/`)

```

WORKING/frame/
├── app/
│   ├── config.py      Model presets (MODEL_WEIGHTS), frozen PipelineConfig dataclass, path
│   ├── detector.py    FaceDetector (YOLO wrapper), FaceDetection dataclass, annotate/crop
│   ├── processing.py  FrameIngestor (sampling), FrameQualityAssessor (6 gates),
│   └── pipeline.py    SINGLE CLI entry – face-pipeline mode, extraction mode, and
│                       run_frame_sampling_quality_layer() used by run_pipeline.py
├── weights/           yolov8n-face-lindevs.pt (auto-downloaded on first use)
├── requirements.txt   ultralytics (YOLO), torch, torchvision (CUDA 12.1), opencv-python
└── README.md

```

Design decision: one CLI entry (`app/pipeline.py`) replaces the legacy `app/main.py` (face pipeline) and `app/extract_frames.py` (extraction) — they no longer exist. Mode is selected automatically by which flags are passed (`--input-dir / --sample-fps /...` ⇒ extraction mode; `--display / --save-metadata /...` ⇒ face-pipeline mode).

4. Pipeline Flow (inside Stage 1)

```

Raw video
├─┬─
│   └─ FrameIngestor → uniform sampling at sample_fps (30 fps in E2E; default 10 fps standalone)
│       ├── timestamp-based sampling using CAP_PROP_POS_MSEC
│       └── automatic fallback to stride sampling if timestamps are unusable (0 ms)
├─┬─
│   └─ FaceDetector → YOLOv8 face detection per sampled frame
│       ├── confidence ≥ 0.35 kept
│       └── device auto-detect (CUDA if available, else CPU), optional FP16
├─┬─
│   └─ FrameQualityAssessor → six rejection rules + composite quality score (0-1)
│       ├── ACCEPTED → JPEG saved to output/frames/frame_sequences/<video>/frames/
│       └── REJECTED → reason(s) recorded (frame itself discarded)
├─┬─
│   └─ Per-frame record appended to frame_metadata.jsonl (accepted AND rejected)
├─┬─
│   └─ Per-video summary → frame_extraction_log.jsonl → aggregate frame_extraction_summary.json
├─┬─
│   └─ rPPG-readiness checks (rppg_input_ready boolean)

```

5. Component Deep-Dives

5.1 FrameIngestor — controlled sampling (`app/processing.py`)

- Source types: local video file, image, stream URL, or webcam index (backends tried in order: DSHOW → MSMF → default).
- Timestamp-based sampling** (preferred): computes `sample_interval_ms = 1000 / sample_fps` and yields a frame whenever the stream timestamp crosses the next target. This produces *uniformly spaced* frames regardless of variable frame duration.
- Stride fallback**: if `CAP_PROP_POS_MSEC` is unusable (returns 0 for the first 2 probe frames), it switches to pure stride sampling (`step = round(source_fps / sample_fps)`) so extraction never stalls on broken timestamps.
- Frames are identified by `frame_id` (sampled index) and `source_frame_id` (original stream index) plus `timestamp_ms` — all preserved in metadata for downstream temporal use.

5.2 FaceDetector — YOLO face detection (`app/detector.py`)

- Uses **YOLOv8 face** weights (lindevs `yolov8n-face-lindevs.pt`, auto-downloaded on first run from GitHub; missing *other* presets raise `FileNotFoundError`).
- Presets: `yolov8n` (default), `yolov8x`, `yolov9e`, `yolov9t`; custom `--weights` override.
- Per frame: runs inference at `--imgsz 320` (default), keeps boxes with confidence $\geq$ `--conf 0.35`, clamps boxes to frame bounds.
- Device: `--device auto` $\Rightarrow$ CUDA if `torch.cuda.is_available()`, else CPU; `--half` enables FP16 only on CUDA.
- Returns face boxes (x1, y1, x2, y2) + confidence; multiple faces per frame are allowed (the largest is used for size checks, max confidence for scoring).

5.3 FrameQualityAssessor — the six rejection gates (`app/processing.py`)

#	Rule	Condition (reject when)	Default	Why it matters for rPPG
1	<code>blur</code>	Laplacian variance < <code>--blur-threshold</code>	8.0	Blur destroys the tiny skin-color pulse signal
2	<code>dark_frame</code>	Gray mean < <code>--dark-threshold</code>	45.0	No illumination $\Rightarrow$ no measurable color change
3	<code>overexposed_frame</code>	Gray mean > <code>--bright-threshold</code>	220.0	Saturated pixels carry no pulsatile variation
4	<code>no_face</code>	No detection above confidence 0.35	—	No face $\Rightarrow$ no ROI to extract pulse from
5	<code>face_too_small</code>	Largest face area ratio < <code>--min-face-area-ratio</code>	0.005	Too few skin pixels per ROI $\Rightarrow$ noisy pulse
6	<code>extreme_pose</code>	Face box touches frame edge margin (3%) or aspect ratio < 0.65 / > 1.7	margin 3%	Cut-off faces break landmark stability

A frame is **ACCEPTED** only if none of the six rules fire (`rejection_reasons` is empty).

5.4 Composite quality score (0–1)

```
quality_score = w1·blur_norm + w2·brightness_norm + w3·face_norm
w1 = 0.4, w2 = 0.3, w3 = 0.3

blur_norm      = clip(Laplacian_var / blur_threshold, 0, 1)
brightness_norm = clip(1 - |gray_mean - 128| / 128, 0, 1)  # mid-tone = best
face_norm      = clip(max_face_confidence, 0, 1)
```

The score is stored per frame in metadata (visualized in the UI) but **does not gate acceptance** — the hard rule gates do.

5.5 StorageManager — asynchronous I/O (app/processing.py)

- Writes JPEGs and JSONL through a `ThreadPoolExecutor` (default 4 workers) so disk I/O never blocks the detection loop; `close()` flushes all pending futures.

6. rPPG-Readiness Checks (per video)

After extraction, the layer computes whether the accepted sequence is actually *usable* by Stage 2 (`summary.rppg_input_ready`):

Check	Condition
<code>sequence_sufficient_for_temporal</code>	accepted frames ≥ <code>--min-seq-len</code> (64)
<code>consistent_frame_resolution</code>	all accepted frames share one resolution
<code>face_resolution_ok</code>	mean accepted face height and width ≥ 48 px
<code>color_variation_ok</code>	std of green-channel mean across accepted frames > 0.5 (pulse must be visible)
<code>sampled_sequence_continuity</code>	inter-frame timestamp deltas within ±35% of the expected interval (drops counted as <code>continuity_breaks</code>)

```
rppg_input_ready = sufficient_for_temporal ^ consistent_resolution ^ face_resolution_ok ^ color_variation_ok
```

If Stage 2 receives too few usable frames anyway (< 48), it returns `features=None` and the pipeline emits **INCONCLUSIVE** (exit code 3) — a designed, honest failure path rather than a guess.

7. Output Contract (artifacts)

Artifact	Path	Content
Accepted frames	<code>output/frames/frame_sequences/<video>/frames/frame_XXXXXX_tYYYYYYYYYY.jpg</code>	JPEGs passing all gates; filename encodes sampled id + timestamp ms
Frame metadata	<code>output/frames/frame_sequences/<video>/frame_metadata.jsonl</code>	One JSON record per sampled frame (accepted or rejected)
Extraction log	<code>output/frames/frame_extraction_log.jsonl</code>	One summary/failure record per video

Artifact	Path	Content
Aggregate summary	output/frames/frame_extraction_summary.json	Per-video summaries incl. rppg_input_ready
Docs	output/frames/docs/	Sampling-rate comparison, quality checklist, quality examples report

Metadata record example

```
{
  "video": "test.mp4",
  "frame_id": 123,
  "source_frame_id": 1230,
  "timestamp_ms": 12345,
  "quality": {
    "accepted": true,
    "quality_flag": "accepted",
    "score": 0.8542,
    "rejection_reasons": [],
    "blur_score": 42.17,
    "brightness_score": 128.3,
    "face_detected": true,
    "face_confidence": 0.9123,
    "max_face_area_ratio": 0.12,
    "face_too_small": false,
    "extreme_pose": false
  },
  "saved_frame": "frame_000123_t0000012345.jpg"
}
```

8. Integration with `run_pipeline.py` (E2E)

```
# WORKING/run_pipeline.py – Stage 1 invocation
run_frame_sampling_quality_layer(
    source=..., weights_path=..., confidence_threshold=0.35,
    image_size=320, device="auto", use_half=False,
    sample_fps=30.0,          # E2E runs at 30 fps – matches the rPPG dataset extraction rate
    min_seq_len=64,
    ...
)
```

- Accepted JPEGs + `frame_metadata.jsonl` are passed to `RPPGPipeline.process_frames()` at the stage-1 sample rate (30 fps).
- Why 30 fps:** rPPG needs the temporal Nyquist resolution for pulse frequencies (0.7–4.0 Hz band); the 10 fps decimation experiment (no anti-alias filter) performed below chance, so 30 fps is the project standard.
- Fallback: if Stage 1 yields no frames, the pipeline falls back to `RPPGPipeline.process_video()` (direct video read) and records which path ran in `input_mode`.

9. Running the Layer

9.1 As part of the full pipeline (recommended)

```
# From WORKING/  
python run_pipeline.py --source path/to/video.mp4 --method POS
```

9.2 Standalone frame stage

```
# From WORKING/frame/  
python app/pipeline.py --source test.mp4 --save-metadata
```

9.3 Extraction mode options

Option	Description	Default
--source	Single video path	required
--input-dir	Directory of videos (batch)	—
--sample-fps	Controlled sampling rate	10.0 (30.0 via run_pipeline.py)
--min-seq-len	Min accepted frames for temporal modeling	64
--blur-threshold	Blur gate	8.0
--dark-threshold	Dark gate	45.0
--bright-threshold	Overexposure gate	220.0
--min-face-area-ratio	Face-size gate	0.005
--conf	Face confidence threshold	0.35
--imgsz	YOLO inference size	320
--device	auto / cpu / gpu / index	auto
--compare-rates	Sampling-rate comparison list for docs	5,10,15
--save-quality-examples	Save one accepted + one rejected example per video	off

10. Real-World Evidence (example run)

Documented sample run (2026-08-19, `Scrape/result_20260819.js`):

Metric	Value
Sampled frames	230
Accepted frames	230 (all passed quality gates)
Rejection distribution	{ } (empty)
Mean quality score	0.956
Mean face confidence	0.862
Temporal coverage ratio	1.0
Resulting rPPG	POS, stage-1-frames input, 230/230 usable frames @ 30 fps

The frame funnel (sampled → accepted → usable rPPG frames) is the layer's key operational metric: **every discarded frame is evidence the pipeline refused to guess on.**

11. Key Engineering Decisions (defense points)

1. **Quality gate before physiology, not after.** Garbage frames are excluded *before* pulse extraction, because a single corrupted frame corrupts the whole trace. Stage 2 no longer re-gates on blur/brightness — Stage 1 already did.
 2. **One CLI, two modes.** `app/pipeline.py` is the single entry point (legacy `main.py` / `extract_frames.py` deleted); `run_frame_sampling_quality_layer()` is the function the E2E orchestrator imports.
 3. **Timestamp-based sampling with stride fallback.** Uniform frame spacing in *time* (not index) is what rPPG needs; the fallback keeps extraction robust on files with broken timestamps.
 4. **30 fps standard.** Matches the rPPG dataset extraction rate; 10 fps decimation (no anti-alias filter) performed below chance — documented in project verification.
 5. **Deterministic acceptance rules.** Six explicit, thresholdable gates with recorded rejection reasons — fully auditable per frame, no black-box scoring gate.
 6. **rPPG-readiness is reported, not assumed.** `rppg_input_ready` + the 48-frame `features=None` / INCONCLUSIVE contract make the *failure* path explicit and safe for KYC.
 7. **YOLOv8 face auto-download** for the default preset (checksum-pinned workflow); other presets fail loudly (`FileNotFoundError`) rather than silently.
-

12. Limitations (honest presentation)

- Very low resolution / heavy compression still challenges the 48×48 px face floor and color-variation check — some KYC captures legitimately fail readiness (designed).
 - Face detection is per-frame (no tracking); detection flicker between frames is tolerated by Stage 2's interpolation, but a tracker is future work.
 - Multi-face frames keep the largest face for size checks — rPPG assumes a single subject.
-

13. Suggested Slide Outline (from this document)

1. **Role of Stage 1** — §1–2 (position in the 3-stage pipeline, why a quality gate matters for rPPG)
 2. **Architecture** — §3–4 (module map, flow diagram)
 3. **Sampling** — §5.1 (30 fps, timestamp-based + fallback; why 30 fps)
 4. **Face detection** — §5.2 (YOLOv8, auto-download, device auto-detect)
 5. **The six quality gates** — §5.3 table (thresholds + rationale)
 6. **Composite score & readiness** — §5.4–6 (formula, `rppg_input_ready` checks)
 7. **Outputs** — §7 (artifacts + metadata schema)
 8. **Integration & E2E** — §8 (`run_pipeline.py` handoff, fallback path)
 9. **Evidence** — §10 (real run funnel: 230/230 accepted, quality 0.956)
 10. **Decisions & limitations** — §11–12
-

Related docs: `README.md` (root), `WORKING/frame/README.md`, `Docs/quantum_layer_guide.md`,
`Docs/Key_Findings_Contributions_Significance.md`, `Docs/RMTT_report.md`.